Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año

MANUAL DE PROGRAMADOR
Mini juego: Piedra, Papel o Tijera en C++

Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año

1. Introducción

Este manual explica cómo está organizado y cómo funciona el código fuente del juego Piedra, Papel o Tijera.
El programa fue desarrollado en C++ y permite jugar contra la computadora, determinar el resultado de cada partida y guardar los datos en un archivo de texto.
Para realizar estas tareas se utilizan estructuras, funciones, números aleatorios y archivos.

2. Organización del proyecto

El proyecto se organiza de la siguiente manera

```
minijuego/
│
├── src/
│   ├── main.cpp
│   └── historial_partidas.txt
│
├── docs/
│   └── manuales/
│
├── capturas/

├── README.md
└── LICENSE
```

El archivo main.cpp contiene el código fuente del programa.
El archivo historial_partidas.txt almacena los resultados de las partidas y se genera cuando el programa guarda una partida, siempre que la ruta sea correcta.
La carpeta docs/manuales contiene la documentación del proyecto.
La carpeta capturas se utiliza para guardar imágenes que demuestren el funcionamiento del juego.

3. Librerías utilizadas

El programa utiliza las siguientes librerías:
iostream
Permite mostrar mensajes en pantalla mediante cout y recibir datos del usuario mediante cin.
fstream
Permite trabajar con archivos, tanto para guardar información como para leerla.
string
Permite utilizar variables de texto, como el nombre del jugador y las elecciones realizadas.
cstdlib
Permite utilizar funciones como rand() y srand(), necesarias para generar la elección aleatoria de la computadora.
ctime
Se utiliza con time(0) para inicializar la semilla de los números aleatorios.

Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año

4. Estructura Partida

El programa utiliza una estructura llamada Partida para agrupar los datos correspondientes a cada partida.
El código es el siguiente:

```
struct Partida {
   int idPartida;
   string jugador;
   string eleccionJugador;
   string eleccionIA;
   string resultado;
};
```

La estructura contiene cinco campos:
idPartida: almacena el número identificador de la partida.
jugador: guarda el nombre o apodo del jugador.
eleccionJugador: almacena la elección realizada por el usuario.
eleccionIA: almacena la elección realizada por la computadora.
resultado: guarda si la partida terminó en victoria, derrota o empate.
La ventaja de utilizar una estructura es que permite organizar todos los datos de una partida en una sola entidad.

5. Función main()

La función main() es el punto de inicio del programa.
Sus principales tareas son:
Inicializar la semilla de los números aleatorios.
Crear el contador de partidas.
Mostrar el menú principal.
Recibir la opción seleccionada por el usuario.
Llamar a la función correspondiente.
Finalizar el programa cuando se selecciona la opción 3.
El menú se repite mediante una estructura do while hasta que el usuario decide salir.

6. Función jugarPartida()

La función jugarPartida() se encarga de desarrollar una partida completa.
Primero, crea una variable de tipo Partida y asigna un identificador.
Después, solicita el nombre del jugador y le permite elegir entre piedra, papel o tijera.
La computadora realiza una elección aleatoria utilizando la siguiente expresión:

```
 int opcIA = (rand() % 3) +1;
```

Esta expresión genera un número entre 1 y 3.
Luego, el programa compara las elecciones y determina el resultado.
Si ambas elecciones son iguales, el resultado es un empate.

Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año

Si el jugador realiza una combinación ganadora, obtiene una victoria. En los demás casos, pierde.
Finalmente, se llama a la función guardarEnDisco() para almacenar los datos de la partida.

7. Función guardarEnDisco()
Esta función se encarga de guardar los datos de una partida en el archivo de texto.
Para abrir el archivo se utiliza:
ofstream archivo("src/historial_partidas.txt", ios::app);

ofstream permite escribir información en un archivo.
Por su parte, ios::app indica que los nuevos datos deben agregarse al final del archivo sin borrar las partidas anteriores.
Los datos se escriben separados por comas:

```
archivo << p.idPartida << ","
        << p.jugador << ","
        << p.eleccionJugador << ","
        << p.eleccionIA << ","
        << p.resultado << endl;
```

Cada partida ocupa una línea del archivo.
Al terminar de escribir, se utiliza archivo.close() para cerrar el archivo.

8. Función leerHistorialDisco()
Esta función permite recuperar las partidas guardadas.
Para abrir el archivo se utiliza:

```
ifstream archivo("src/historial_partidas.txt");
```

ifstream permite leer información almacenada en un archivo.
El programa utiliza getline() para recuperar los campos separados por comas.
Después, muestra en pantalla el número de partida, el nombre del jugador, las elecciones realizadas y el resultado.
También utiliza un contador para indicar cuántas partidas se leyeron.
Si el archivo no existe o no puede abrirse, el programa muestra un mensaje de aviso.
9. Persistencia de datos
La persistencia de datos permite conservar información aunque el programa se cierre.
En este proyecto, las partidas se almacenan en historial_partidas.txt.
Para lograrlo, se utilizan dos herramientas principales:
ofstream: permite escribir y guardar las partidas.
ifstream: permite leer las partidas guardadas.
Además, se utiliza ios::app para agregar nuevas partidas sin sobrescribir los registros anteriores.
Gracias a este sistema, el historial puede consultarse cada vez que se ejecuta el programa, siempre que se mantenga el archivo en la ubicación correcta.

Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año

10. Compilación y ejecución

Para compilar el programa desde la carpeta principal del proyecto, se utiliza el siguiente comando:
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
Para ejecutarlo, se utiliza:
.\src\piedrapapelotijera.exe
Estos comandos corresponden a un proyecto donde main.cpp está dentro de la carpeta src. Si el archivo se encuentra directamente en la carpeta principal, es necesario adaptar los comandos a su ubicación.

11. Pruebas del programa

Para comprobar que el programa funciona correctamente, se recomienda realizar las siguientes pruebas:
Ejecutar el programa y comprobar que aparece el menú principal.
Jugar una partida y verificar que se muestra el resultado.
Comprobar que se crea el archivo historial_partidas.txt.
Jugar varias partidas y verificar que se guardan todas.
Seleccionar la opción 2 para consultar el historial.
Cerrar el programa y volver a ejecutarlo.
Consultar nuevamente el historial y comprobar que las partidas anteriores siguen guardadas.
Estas pruebas permiten verificar el funcionamiento del juego y la persistencia de los datos.

12. Posibles mejoras

En futuras versiones se podrían realizar algunas mejoras:
Validar las entradas del usuario para evitar errores.
Recuperar el número de la última partida para continuar la numeración después de cerrar el programa.
Mejorar el formato del archivo de historial.
Agregar estadísticas de victorias, derrotas y empates.
Incorporar un menú para consultar los resultados de un jugador específico.

13. Conclusión

El desarrollo de este programa permite aplicar conceptos fundamentales de C++, como estructuras, funciones, condiciones, números aleatorios y manejo de archivos.
Además, permite comprender la diferencia entre la memoria RAM y el almacenamiento en disco.
La utilización de struct, ofstream, ifstream e ios::app hace posible organizar los datos de cada partida, guardarlos y recuperarlos posteriormente.
El proyecto demuestra cómo se puede implementar un sistema sencillo de persistencia de datos en una aplicación desarrollada en C++.

Estudiante: Alma Olivola
Materia: Laboratorio de Programación
Curso: 5.º año